Conversioni di tipo

Marco Cerbella

Lezione 10 – 13 ottobre 2026

1 Conversioni

Operandi di tipo diverso si possono combinare in una stessa operazione.

```
1 double dVar = 2.5;
2 dVar = dVar * 3;
3 if (dVar < 10L) { /* ... */ }
```

Quando gli operandi hanno tipi diversi, il compilatore cerca di convertirli a un tipo uniforme prima di eseguire l'operazione.

1.1 Conversioni implicite

Il compilatore applica conversioni di tipo implicite quando

- gli operandi hanno tipi non corrispondenti;
- si chiama una funzione con un argomento il cui tipo non corrisponde a quello del parametro;
- si inizializzano variabili o si assegnano loro valori.

Se la conversione necessaria non è possibile il compilatore segnala un errore; altre volte segnala un warning.

1.2 Operatore di cast

Si può convertire un valore da un tipo a un altro anche *esplicitamente*, con l'operatore di cast:

`(nome_tipo) espressione`

```
1 int sum = 10, count = 3;
2 double mean = (double) sum / count;   // 3.3333
```

Il valore di `sum` viene prima convertito (esplicitamente) in `double`; il compilatore deve poi convertire implicitamente il divisore, il valore di `count`, allo stesso tipo prima di eseguire la divisione.

Osservazione. Conviene usare sempre il cast quando c'è la possibilità di perdere informazione. I cast espliciti evitano i warning del compilatore.

2 Conversione dei tipi aritmetici

2.1 Gerarchia dei tipi

Quando gli operandi aritmetici hanno tipi diversi, la conversione implicita è governata dal *rango di conversione* dei tipi:

- due tipi interi senza segno qualsiasi hanno ranghi diversi: se uno è più largo dell'altro ha rango più alto;
- ogni tipo intero con segno ha lo stesso rango del corrispondente tipo senza segno;
- i tipi interi standard sono ordinati: `_Bool` < `char` < `short` < `int` < `long` < `long long`;
- i tipi in virgola mobile sono ordinati: `float` < `double` < `long double`;
- il tipo in virgola mobile di rango più basso, `float`, ha rango più alto di qualunque tipo intero;
- gli `enum` hanno lo stesso rango di `int`.

2.2 Promozione intera

In qualunque espressione si può usare un valore di tipo con rango inferiore a `int` al posto di un operando di tipo `int` o `unsigned int`. In questi casi il compilatore applica la *promozione intera*: ogni operando di rango inferiore a `int` viene convertito automaticamente in `int`, purché `int` sia in grado di rappresentare tutti i valori del tipo originale; altrimenti viene convertito in `unsigned int`. Le operazioni nella CPU vengono eseguite su almeno 4 byte.

Esempio.

```c
#include <stdio.h>
int main()
{
    char a = 30, b = 40, c = 10;
    char d = (a * b) / c;
    printf("%d ", d);
    return 0;
}
```

```
120
```

A prima vista `(a*b)/c` sembra causare un overflow aritmetico: i `char` con segno vanno da -128 a 127 (nella maggior parte dei compilatori) e il valore di `(a*b)` è 1200. Ma sui tipi `char` avviene la promozione intera, quindi si ottiene il risultato corretto senza overflow.

2.3 Conversioni aritmetiche usuali

Le *conversioni aritmetiche usuali* sono le conversioni implicite applicate automaticamente agli operandi di tipo aritmetico diverso per la maggior parte degli operatori. Si applicano implicitamente a

- gli operatori aritmetici a due operandi: `*`, `/`, `%`, `+` e `-`;
- gli operatori relazionali e di uguaglianza: `<`, `<=`, `>`, `>=`, `==` e `!=`.

Esempio.

```c
int x = 0;
int i = -1;
unsigned int limit = 200U;
long n = 30L;

if (i < limit)
    x = limit * n;

printf("%d\n", x);
```

```
0
```

Le conversioni aritmetiche usuali si applicano così:

- Se uno degli operandi ha tipo in virgola mobile, l'operando di rango più basso viene convertito a un tipo con lo stesso rango dell'altro. I tipi reali si convertono solo in tipi reali.
- Se entrambi gli operandi sono interi, si esegue prima la promozione intera su entrambi. Se dopo la promozione i tipi sono ancora diversi, si procede così:
  - **Regola 1**: se un operando ha un tipo *senza segno* T con rango almeno pari a quello del tipo dell'altro operando, l'altro operando viene convertito al tipo senza segno T.
  - **Regola 2**: altrimenti, un operando ha un tipo *con segno* T con rango più alto di quello dell'altro. L'altro operando viene convertito al tipo con segno T solo se T è in grado di rappresentare tutti i valori del suo tipo precedente; se non lo è, entrambi gli operandi vengono convertiti al tipo senza segno corrispondente a T.

Esempio (Applicazione delle regole)**.** Nell'esempio sopra, per valutare il confronto `i < limit` il valore di `i`, -1, viene convertito a `unsigned int` (Regola 1: `limit` è senza segno e ha rango almeno pari a `int`). Il risultato è un numero positivo molto grande, quindi la condizione dell'`if` è *falsa* e `x` resta 0.

In `limit * n` (Regola 2) `limit` viene convertito al tipo di `n`, `long`, se l'intervallo dei valori di `long` contiene tutto l'intervallo di `unsigned int`. Altrimenti (ad esempio se `int` e `long` hanno entrambi 32 bit) entrambi i fattori vengono convertiti a `unsigned long`.

Come si rappresenta -1 in un `int` (little endian, complemento a due)? Con 32 bit tutti a 1: `11111111111111111111111111111111`. Un `unsigned int` ha 32 bit (in quel compilatore), quindi -1 viene convertito implicitamente in `unsigned int` e il suo valore diventa 4 294 967 295.

3 Altre conversioni implicite

Il compilatore converte automaticamente i valori aritmetici anche nei casi seguenti:

- nelle assegnazioni e nelle inizializzazioni, il valore dell'operando di destra viene sempre convertito al tipo dell'operando di sinistra;
- nelle chiamate di funzione, gli argomenti vengono convertiti ai tipi dei parametri corrispondenti;

- nelle istruzioni `return`, il valore dell'espressione viene convertito al tipo restituito dalla funzione.

In un'*assegnazione composta*, come `x += 2.5` (cioè `x = x + 2.5`), i valori di entrambi gli operandi sono prima soggetti alle conversioni aritmetiche usuali; poi il risultato dell'operazione viene convertito, come in una assegnazione semplice, al tipo dell'operando di sinistra. Se `x` è `int`, viene convertito in `double` e poi il risultato `x + 2.5` (di tipo `double`) viene riconvertito in `int`.

Esempio.

```
#include <math.h>   // dichiara double sqrt(double)
int i = 7;
float x = 0.5;      // la costante viene convertita da double a float

i = x;              // il valore di x viene convertito da float a int

x += 2.5;           // prima dell'addizione x viene convertito in double;
                    // la somma viene poi convertita in float per l'assegnazione

x = sqrt(i);        // l'argomento viene convertito da int a double;
                    // il valore restituito da double a float

long my_func() {    // la costante 0 viene convertita a long,
    /* ... */       // il tipo restituito dalla funzione
    return 0;
}
```

4 Conversioni verso tipi interi

4.1 Verso tipi interi senza segno

I valori interi sono sempre preservati se rientrano nell'intervallo del nuovo tipo senza segno, cioè tra 0 e `Utype_MAX`. Per i valori fuori intervallo, il valore dopo la conversione si ottiene sommando o sottraendo `Utype_MAX` + 1 tante volte quante servono perché il risultato rientri nell'intervallo del nuovo tipo.

```
unsigned short n = 1000;   // 1000 rientra nell'intervallo di unsigned short: ok
n = -1;                    // il valore -1 deve essere convertito
```

Si ha $-1 + (\texttt{USHRT_MAX} + 1) = \texttt{USHRT_MAX}$, quindi l'ultima istruzione equivale a `n = USHRT_MAX;`.

4.2 Da virgola mobile a interi

Per convertire un numero reale in virgola mobile in un tipo intero (con o senza segno) il compilatore scarta la parte frazionaria. Se la parte intera rimanente è fuori dall'intervallo del nuovo tipo, il risultato della conversione è *indefinito*.

```
double x = 2.9;
unsigned long n = x;   // la parte frazionaria di x viene persa: n = 2
```

4.3 Verso tipi interi con segno

Il problema del superamento dell'intervallo dei valori del tipo di destinazione può presentarsi anche quando un valore intero, con o senza segno, viene convertito in un tipo intero *con segno*, ad esempio da `long` o `unsigned int` a `int`. A differenza delle conversioni verso tipi senza segno, il risultato di un overflow in conversione verso un tipo con segno è lasciato all'implementazione. La maggior parte dei compilatori scarta i bit più significativi della rappresentazione binaria del valore originale e interpreta i bit più bassi secondo il nuovo tipo.

Esempio.

```c
#include <stdio.h>
#include <limits.h>
int main() {
    long long int a = (LLONG_MAX - UINT_MAX) + 1;
    int b = a;

    printf("b: %d\n", b);
    printf("a: %lld\n", a);
}
```

```
b: 1
a: 9223372032559808513
```

In esadecimale `LLONG_MAX` è `0x7FFFFFFFFFFFFFFF` e `UINT_MAX` è `0xFFFFFFFF`, quindi

$$a = \texttt{0x7FFFFFFF00000001}.$$

Convertendo in `int` (32 bit) si tengono solo i 32 bit più bassi, `00000001`: perciò $b = 1$.

5 Conversioni verso tipi in virgola mobile

Non tutti i valori interi sono rappresentabili esattamente nei tipi in virgola mobile. Ad esempio, benché l'intervallo di valori del tipo `float` includa quello di `long` e `long long`, `float` ha una precisione di sole sei cifre decimali. Quindi alcuni valori `long` non possono essere memorizzati esattamente in un `float`: il risultato della conversione è il valore rappresentabile immediatamente inferiore o superiore.

Esempio.

```c
float r_var = 16777217;
double l_var = 16777217;
printf("The rounding error (l_var - r_var) is %.2f\n", l_var - r_var);
printf("r_var is %.2f\n", r_var);
```

```
The rounding error (l_var - r_var) is 1.00
r_var is 16777216.00
```

Ogni valore di un tipo in virgola mobile si può rappresentare esattamente in un altro tipo in virgola mobile di precisione maggiore. Quindi, quando un `double` viene convertito in `long double`, o un `float` in `double` o `long double`, il valore è esattamente preservato.

Nelle conversioni da un tipo più preciso a uno meno preciso, invece, il valore da convertire può essere fuori dall'intervallo del nuovo tipo.

- Se il valore supera l'intervallo del tipo di destinazione, il risultato della conversione è *indefinito*.
- Se il valore è nell'intervallo ma non è rappresentabile esattamente con la precisione del tipo di destinazione, il risultato è il valore rappresentabile immediatamente inferiore o superiore.

Riferimenti

Niente sul libro. Risorse indicate dal docente:

- `https://www.w3schools.com/c/c_type_conversion.php`
- `https://www.guru99.com/c-type-casting.html`
- soprattutto, il capitolo 4 di *C in a Nutshell* (O'Reilly): `https://www.oreilly.com/library/view/c-in-a/0596006977/ch04.html`